

# M2S-06: Step 6 — Integrate: Reconnect Integrations

**Series:** M2S — Managed to SaaS Migration | **Notebook:** 6 of 9 | **Phase:** Upgrade | **Step:** Integrate | **Created:** March 2026 | **Last Updated:** 08/12/2026

With OneAgents reporting to SaaS and configurations migrated, the next challenge is ensuring every external system that depends on Dynatrace is reconnected. Dashboards need updated links, alerting channels need validation, CI/CD pipelines need new API endpoints, and ITSM integrations need reconfiguration. This notebook provides a systematic approach to reconnecting every integration point.

**M2S Migration Journey — 3 Phases / 9 Steps**

**Plan:** 1. Discover | 2. Strategize | 3. Design

**Upgrade:** 4. Prepare | 5. Execute | **6. Integrate**

**Run:** 7. Enable | 8. Expand | 9. Optimize

## Table of Contents

- 1. [Dashboard Integration](#)
- 2. [Alerting and Notification Integration](#)
- 3. [CI/CD Pipeline Integration](#)
- 4. [ITSM and Incident Management](#)
- 5. [Extension Migration](#)
- 6. [API Script Migration](#)
- 7. [Synthetic Monitor Migration](#)
- 8. [Step Completion Checklist](#)

## Prerequisites

| Requirement                | Details                                                                 |
|----------------------------|-------------------------------------------------------------------------|
| Steps 1-5 Complete         | Discovery, strategy, design, preparation, and execution phases finished |
| SaaS Tenant Active         | OneAgents reporting data to SaaS environment                            |
| SaaS API Tokens            | Tokens with appropriate scopes for each integration                     |
| Integration Inventory      | List of all third-party integrations from Step 1 (Discover)             |
| Access to External Systems | Admin access to Slack, Teams, PagerDuty, ServiceNow, CI/CD tools        |

**Order of Operations — You Are Here**

This notebook covers operations **7-8** of the 11-step Order of Operations (see **M2S-02: Step 2 — Strategize**) defined in Step 2:

| #  | Operation                                          | Status                                                                                              |
|----|----------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 1  | Assess — Inventory current environment             | Step 4: Prepare  |
| 2  | Provision — SaaS tenant and access (SSO)           | Step 4: Prepare  |
| 3  | Install — New ActiveGates in parallel              | Step 4: Prepare  |
| 4  | Migrate — Configuration and integrations           | Step 5: Execute  |
| 5  | Rebuild — Dashboards, zones, alerts                | Step 5: Execute  |
| 6  | Redirect — OneAgents to SaaS                       | Step 5: Execute  |
| 7  | <b>Reconnect — Integrations and extensions</b>     | <b>This notebook</b>                                                                                |
| 8  | <b>Migrate — Remaining config and integrations</b> | <b>This notebook</b>                                                                                |
| 9  | Validate — Data flow and performance               | Step 9: Optimize                                                                                    |
| 10 | Cutover — Full switch to SaaS                      | Step 9: Optimize                                                                                    |
| 11 | Decommission — Managed environment                 | Step 9: Optimize                                                                                    |

## 1. Dashboard Integration

Dashboards migrated via the SaaS Upgrade Assistant retain their structure, but several elements need manual attention. Internal links, entity references, and ownership must be validated against the new SaaS environment.

### What Needs Updating

| Item                           | Why                                          | Action                                                                                                                     |
|--------------------------------|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Dashboard ownership</b>     | Managed user IDs differ from SaaS user IDs   | Reassign ownership via SaaS Upgrade Assistant bulk edit or manually                                                        |
| <b>Internal links</b>          | URLs pointing to Managed UI pages are broken | Update all <code>https://{managed-url}/e/{env-id}/...</code> links to <code>https://{tenant}.apps.dynatrace.com/...</code> |
| <b>Management zone filters</b> | MZ IDs may differ between environments       | Verify MZ filters resolve correctly; update IDs if needed                                                                  |
| <b>Hardcoded entity IDs</b>    | Managed entity IDs may not match SaaS        | Replace hardcoded IDs with entity selectors where possible                                                                 |
| <b>Tile data sources</b>       | Some tiles may reference unavailable data    | Verify every tile renders data                                                                                             |

### Dashboard Validation Strategy

Work through dashboards in priority order:

1. **Executive dashboards** — High visibility, first to be noticed if broken
2. **Operations dashboards** — Used for daily monitoring decisions
3. **Team-specific dashboards** — Application and service team views
4. **Report dashboards** — Used for periodic reporting

### Verify Dashboards Are Present

Dashboards are **Document Service** objects on SaaS, not Grail entities — `fetch dt.entity.dashboard` is **not valid DQL** and returns no data. To list and count dashboards on your SaaS tenant so you can compare against your Managed inventory, use the **Document Service API**:

```
# Count all dashboards migrated to SaaS (compare to your Managed inventory)
curl -s "https://{tenant}.apps.dynatrace.com/platform/document/v1/documents?filter=type='dashboard'" \
-H "Authorization: Api-Token {TOKEN}" | jq '.totalCount'

# List dashboard names and owners
curl -s "https://{tenant}.apps.dynatrace.com/platform/document/v1/documents?filter=type='dashboard'" \
-H "Authorization: Api-Token {TOKEN}" | jq -r '.documents[] | "\(.name)\t\(.owner)"'
```

Requires the `documents:read` token scope. Alternatively, open the **Dashboards** app in the SaaS UI and compare the list against your Managed dashboard inventory.

### URL Format Changes

Every internal link in dashboards must be updated from Managed format to SaaS format:

| Page            | Managed URL                                                       | SaaS URL                                                                 |
|-----------------|-------------------------------------------------------------------|--------------------------------------------------------------------------|
| Host details    | <code>https://{managed}/e/{env-id}/#host;id={HOST-ID}</code>      | <code>https://{tenant}.apps.dynatrace.com/ui/entity/{HOST-ID}</code>     |
| Service details | <code>https://{managed}/e/{env-id}/#service;id={SVC-ID}</code>    | <code>https://{tenant}.apps.dynatrace.com/ui/entity/{SVC-ID}</code>      |
| Dashboard link  | <code>https://{managed}/e/{env-id}/#dashboard;id={DASH-ID}</code> | <code>https://{tenant}.apps.dynatrace.com/ui/dashboards/{DASH-ID}</code> |
| Notebook        | N/A (Managed)                                                     | <code>https://{tenant}.apps.dynatrace.com/ui/document/{DOC-ID}</code>    |

**Tip:** Use the Dashboard API to programmatically search for and replace Managed URLs across all dashboards. This is faster and more reliable than manual editing for large dashboard inventories.

### Replacing Hardcoded Entity IDs

Where dashboards use hardcoded entity IDs in tile filters, replace them with entity selectors:

| Before (Managed)                        | After (SaaS)                                                                  |
|-----------------------------------------|-------------------------------------------------------------------------------|
| <code>entityId: "HOST-ABC123"</code>    | <code>entitySelector: type(HOST), tag("environment:production")</code>        |
| <code>entityId: "SERVICE-DEF456"</code> | <code>entitySelector: type(SERVICE), entityName.startsWith("checkout")</code> |

Entity selectors are resilient to entity ID changes and work across environments.

## 2. Alerting and Notification Integration

Alerting is the most critical integration to validate. A missed alert during migration can mean an undetected production incident. Every notification channel must be tested end-to-end before declaring the migration complete.

## What Changes Between Managed and SaaS

| Component                     | Managed                                       | SaaS                                                    |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| <b>Problem notifications</b>  | Classic notification rules                    | Migrate to SaaS notification rules or Workflow triggers |
| <b>Webhook source URL</b>     | <code>https://{managed}/e/{env-id}/...</code> | <code>https://{tenant}.apps.dynatrace.com/...</code>    |
| <b>Alerting profiles</b>      | Migrated via SaaS Upgrade Assistant           | Verify filters and thresholds match                     |
| <b>Notification templates</b> | May reference Managed-specific variables      | Update template variables for SaaS                      |

## Integration Endpoints to Update

| Integration            | What Changes                                          | What Stays the Same          |
|------------------------|-------------------------------------------------------|------------------------------|
| <b>Slack</b>           | Dynatrace source URLs in message payloads             | Webhook URL                  |
| <b>Microsoft Teams</b> | Dynatrace source URLs in adaptive cards               | Webhook URL                  |
| <b>PagerDuty</b>       | Create new integration in SaaS; Dynatrace source URLs | Routing key                  |
| <b>ServiceNow</b>      | Create new Dynatrace integration user in SNOW         | ServiceNow instance URL      |
| <b>Custom webhooks</b> | All Dynatrace API endpoints in webhook payloads       | External system endpoints    |
| <b>Email</b>           | Notification sender address (Dynatrace SaaS domain)   | Recipient distribution lists |

## Converting to Workflow-Based Notifications

SaaS provides the AutomationEngine for advanced alerting workflows. Consider converting Managed problem notifications to Workflow triggers for:

- **Conditional routing** — Route alerts based on management zone, severity, or tags
- **Enrichment** — Add entity details, runbook links, or Dynatrace Intelligence context to notifications
- **Multi-channel delivery** — Send to Slack AND PagerDuty from a single workflow
- **Auto-remediation** — Trigger runbooks or API calls in response to problems

## End-to-End Alert Validation

For each notification channel:

1. **Configure** — Set up the notification integration in SaaS
2. **Trigger** — Create a test problem or use a custom event to fire an alert
3. **Verify delivery** — Confirm the alert arrives at the destination
4. **Check content** — Verify URLs, entity names, and context are correct
5. **Test resolution** — Close the problem and verify the resolution notification

**Warning:** Do not skip testing resolution notifications. Many integrations (PagerDuty, ServiceNow) auto-resolve incidents when the Dynatrace problem closes. If resolution notifications are broken, incidents pile up in your ITSM system.

## Verify Alert Delivery Using Detected Problems

After configuring notifications, monitor detected problems to confirm alerts are flowing:

```
// Recent problems – verify these triggered notifications in your external systems
fetch dt.davis.problems, from:-24h
| fieldsKeep display_id, event.name, event.status, event.category, timestamp
| sort timestamp desc
| limit 20
```

```
// Problem trend over last 7 days – confirms Dynatrace Intelligence baseline is building
fetch dt.davis.problems, from:-7d
| makeTimeseries count = count(default: 0), interval: 24h
```

### 3. CI/CD Pipeline Integration

Every CI/CD pipeline that integrates with Dynatrace needs its connection parameters updated. This includes deployment event notifications, quality gate evaluations, and release tracking.

#### URL Format Change

All Dynatrace API calls must switch from Managed format to SaaS format:

| API                  | Managed Format                                              | SaaS Format                                                        | Survives the Gen3 upgrade?                                        |
|----------------------|-------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------|
| Environment API v2   | <code>https://{managed}/e/{env-id}/api/v2/...</code>        | <code>https://{tenant}.live.dynatrace.com/api/v2/...</code>        | Ingest paths yes; several read/config paths <b>no</b> — see below |
| Configuration API v1 | <code>https://{managed}/e/{env-id}/api/config/v1/...</code> | <code>https://{tenant}.live.dynatrace.com/api/config/v1/...</code> | <b>Mostly no</b>                                                  |
| Platform API         | N/A (Managed)                                               | <code>https://{tenant}.apps.dynatrace.com/platform/...</code>      | Yes                                                               |

**Repoint once, not twice.** This step is a host-and-path rewrite for the Managed → SaaS move, and on that axis the table above is correct. But a separate change is coming: when the tenant later upgrades to the latest Dynatrace, **most of /api/config/v1 and a number of /api/v2 read and configuration paths stop answering**, while the **ingest endpoints** ( /api/v2/logs/ingest , /metrics/ingest , /events/ingest , /api/v2/otlp , /api/bizevents/ingest ) and everything under /platform/ carry forward.

The narrow /api/config/v1 carve-outs that do survive are worth knowing, because they are easy to confuse with neighbours that do not: /service/requestAttributes , /service/requestNaming and /service/customServices survive, while /conditionalNaming/\* and /calculatedMetrics/service do not — and /calculatedMetrics/mobile survives while /calculatedMetrics/service does not.

If a script is being rewritten anyway, check it against AUTOM-02's catalog now and move it to its Gen3 equivalent in the same pass rather than migrating it twice. See the Classic → Gen3 doorway in the -START-HERE- playbook for the sequence.

## CI/CD Tools to Update

| Tool           | Configuration to Update                                                           | Where to Change                                                   |
|----------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Jenkins        | Dynatrace plugin configuration, API token                                         | Jenkins system configuration and pipeline credentials             |
| GitHub Actions | Repository secrets: <code>DYNATRACE_URL</code> , <code>DYNATRACE_API_TOKEN</code> | Repository Settings > Secrets and Variables                       |
| Azure DevOps   | Service connections, pipeline variables                                           | Project Settings > Service Connections                            |
| GitLab CI      | CI/CD variables: <code>DT_URL</code> , <code>DT_TOKEN</code>                      | Settings > CI/CD > Variables                                      |
| Ansible        | Dynatrace collection connection variables                                         | Inventory or playbook variables                                   |
| Terraform      | Provider configuration ( <code>environment_url</code> , <code>api_token</code> )  | Provider block in <code>.tf</code> files or environment variables |

## Generate New API Tokens

Create dedicated API tokens for each CI/CD integration. Follow the principle of least privilege:

| Integration        | Required Token Scopes                                    |
|--------------------|----------------------------------------------------------|
| Deployment events  | <code>events.ingest</code>                               |
| Configuration push | <code>settings.write</code> , <code>settings.read</code> |
| Entity query       | <code>entities.read</code>                               |
| Metric ingestion   | <code>metrics.ingest</code>                              |
| SLO evaluation     | <code>slo.read</code>                                    |
| Full automation    | Combine scopes per tool requirements                     |

**Important:** Never reuse a single token across all CI/CD tools. If one token is compromised, all integrations are affected. Create one token per tool with only the scopes that tool requires.

## Deployment Event Validation

After updating CI/CD configurations, trigger a test deployment and verify that deployment events appear in SaaS:

```
// Verify deployment events are being received from CI/CD pipelines
fetch events, from:-24h
| filter event.kind == "CUSTOM_DEPLOYMENT"
| fieldsKeep event.type, dt.entity.service, timestamp
| sort timestamp desc
| limit 10
```

## Release Tracking

If you use Dynatrace release tracking, verify that version metadata is being captured correctly after the CI/CD update:

```
// Check release versions captured by Dynatrace via deployment events
// Release version and stage are carried in deployment events, not as entity fields
fetch events, from:-7d
| filter event.kind == "CUSTOM_DEPLOYMENT"
| fieldsKeep event.name, dt.entity.service, event.deployment.release_version, event.deployment.release_stage,
timestamp
| sort timestamp desc
| limit 20

// Note: releaseVersion / releaseStage are not directly addressable fields on dt.entity.service in DQL.
// Release tracking data surfaces through deployment events (event.kind == "CUSTOM_DEPLOYMENT") or
// via the Release Monitoring app in the Dynatrace UI.
```

## 4. ITSM and Incident Management

ITSM integrations are business-critical — they drive incident response workflows. A broken ITSM integration means problems detected by Dynatrace never reach the teams that can fix them.

### ServiceNow Integration

| Configuration Item     | Action                                                                      |
|------------------------|-----------------------------------------------------------------------------|
| Dynatrace instance URL | Update from Managed URL to <code>https://{tenant}.live.dynatrace.com</code> |
| Integration user       | Create new integration user with SaaS API token                             |
| Problem webhook        | Reconfigure webhook to send from SaaS                                       |
| CMDB synchronization   | Update API endpoint for entity data push                                    |
| Event management       | Verify event forwarding from SaaS to ServiceNow event management            |

### Jira Integration

| Configuration Item        | Action                                              |
|---------------------------|-----------------------------------------------------|
| Webhook endpoint          | Update Dynatrace webhook to call Jira from SaaS     |
| Problem-to-ticket mapping | Verify field mappings work with SaaS problem format |
| Back-link URLs            | Update Jira ticket templates to link to SaaS URLs   |

### CMDB Synchronization

If you synchronize Dynatrace entities to a CMDB:

1. **Update API endpoint** — Point CMDB sync to SaaS Entities API v2
2. **Update authentication** — New SaaS API token with `entities.read` scope
3. **Verify entity mapping** — Entity IDs may differ; use entity selectors or tags for matching
4. **Test sync cycle** — Run a sync and confirm entities appear correctly in the CMDB

Validate ITSM Connectivity

```
// Check for recent closed problems – use these to verify ITSM tickets were created and resolved
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| fieldsKeep display_id, event.name, event.category, timestamp
| sort timestamp desc
| limit 10
```

5. Extension Migration

Extensions provide monitoring for technologies not covered by OneAgent auto-instrumentation. The extension framework has changed significantly between Managed and SaaS.

Extension Framework Comparison

| Framework             | Managed   | SaaS                           | Action                                     |
|-----------------------|-----------|--------------------------------|--------------------------------------------|
| Extensions 1.0 (EF1)  | Supported | Deprecated — limited support   | Rebuild as Extensions 2.0                  |
| Extensions 2.0 (EF2)  | Supported | Fully supported                | Migrate configuration; may need re-signing |
| ActiveGate extensions | Supported | Supported (host-based AG only) | Deploy on host-based ActiveGates in SaaS   |
| Custom plugins        | Supported | Not supported                  | Rebuild as Extensions 2.0                  |

Extensions 2.0 Requirements in SaaS

| Requirement              | Details                                                                                             |
|--------------------------|-----------------------------------------------------------------------------------------------------|
| Host-based ActiveGate    | Extensions 2.0 require a host-based ActiveGate — Kubernetes-based ActiveGates cannot run extensions |
| Extension signing        | Extensions must be signed with a valid certificate for SaaS                                         |
| Credential Vault         | Extension credentials must be re-entered in SaaS Credentials Vault                                  |
| Monitoring configuration | Extension monitoring configurations need recreation in SaaS                                         |

Cloud Integration Verification

Cloud platform integrations (AWS, Azure, GCP) require reconfiguration in SaaS because credentials are not portable:

| Cloud Platform | Configuration to Update                                      |
|----------------|--------------------------------------------------------------|
| AWS            | IAM role or access key for CloudWatch metrics and ActiveGate |
| Azure          | Service principal credentials for Azure Monitor              |
| GCP            | Service account key for Google Cloud Monitoring              |

After reconfiguring cloud integrations, verify metrics are flowing:



```
// Verify AWS cloud metrics are flowing after integration reconfiguration
//
// The `dt.cloud.aws.*` metric namespace DOES NOT EXIST in any spelling (verified 08/12/2026).
// AWS CloudWatch metrics are published as
cloud.aws.&lt;service&gt;.&lt;CloudWatchName&gt;.By.&lt;Dimension&gt;; -
// the CloudWatch name keeps its PascalCase and the split dimension is part of the key. A
// timeseries against a missing key returns an EMPTY result rather than an error, so this cell
// silently drew nothing. Enumerate with:
//   metrics | filter startsWith(metric.key, "cloud.aws") | fields metric.key | sort metric.key asc
timeseries cpu = avg(cloud.aws.ec2.CPUUtilization.By.InstanceId), from:-1h, by:{InstanceId}
| fieldsAdd avg_cpu = arrayAvg(cpu)
| fields InstanceId, avg_cpu
| sort avg_cpu desc
| limit 5
```

```
// Verify Azure cloud metrics are flowing
//
// Corrected 08/12/2026: `dt.cloud.azure.vm.cpu_usage` does not exist. The `dt.cloud.azure.*`
// namespace is real but carries only VM *counts* (running / stopped / initializing, by region and
// scale set) – there is no CPU metric in it. Azure Monitor resource metrics are published as
// cloud.azure.&lt;provider_namespace&gt;.&lt;resource_type&gt;.&lt;MetricName&gt;;, so VM CPU is
// PercentageCPU.
// (For OneAgent-monitored Azure VMs, dt.host.cpu.usage is the host-level equivalent.) Enumerate:
//   metrics | filter startsWith(metric.key, "cloud.azure") | fields metric.key | sort metric.key asc
timeseries cpu = avg(cloud.azure.microsoft_compute.virtualmachines.PercentageCPU), from:-1h
| fieldsAdd avg_cpu = arrayAvg(cpu)
| fields avg_cpu
```

## Extension Health Check

```
// Check ActiveGate health – extensions run on host-based ActiveGates carrying the
// EXTENSION_CONTROLLER module, so filter to exactly those rather than listing every AG
smartscapeNodes "ACTIVEGATE"
| filter is_containerized == false and iAny(modules[] == "EXTENSION_CONTROLLER")
| fields name, version = dt.active_gate.version, group = dt.active_gate.group.name, modules
| sort name asc

// Correction (verified 07/2026): this cell previously ran `fetch dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong. There is NO classic
// ActiveGate entity type in any spelling (active_gate, environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every tenant –
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes "ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
// is_containerized and modules are Smartscape-only fields – the classic entity never exposed them,
// so this host-based-AG filter was not expressible before.
```

## 6. API Script Migration

Most organizations have scripts and tools that call Dynatrace APIs for automation, reporting, or meta-monitoring. Every one of these must be updated.

### URL Format Change

| API Endpoint         | Managed Format                                 | SaaS Format                                           |
|----------------------|------------------------------------------------|-------------------------------------------------------|
| Environment API v2   | https://{managed}/e/{env-id}/api/v2/...        | https://{tenant}.live.dynatrace.com/api/v2/...        |
| Configuration API v1 | https://{managed}/e/{env-id}/api/config/v1/... | https://{tenant}.live.dynatrace.com/api/config/v1/... |
| Cluster API          | https://{managed}/api/cluster/v2/...           | N/A — use Account Management API instead              |

**Important:** The Cluster Management API does not exist in SaaS. Scripts that use cluster-level APIs (user management, environment provisioning, license management) must be rewritten to use the [Account Management API](#) or the SaaS UI.

### Common Script Types to Audit

| Script Type            | Typical Purpose                           | Update Required                                                  |
|------------------------|-------------------------------------------|------------------------------------------------------------------|
| Health check scripts   | Monitor Dynatrace cluster availability    | Update URL and token; remove cluster-specific checks             |
| Reporting scripts      | Pull metrics and SLO data for reports     | Update URL and token                                             |
| Provisioning scripts   | Create management zones, tags, dashboards | Update URL and token                                             |
| Token rotation scripts | Rotate API tokens automatically           | Rewrite for SaaS token management                                |
| Backup scripts         | Export configurations periodically        | Update URL and token                                             |
| Meta-monitoring        | Monitor the monitoring platform itself    | Update URL and token; adjust health checks for SaaS architecture |

### Token Migration Strategy

- 1. **Inventory all tokens** — List every API token used in Managed with its purpose and scopes
- 2. **Create SaaS equivalents** — Generate new tokens in SaaS with matching (or tighter) scopes
- 3. **Update scripts** — Replace Managed tokens with SaaS tokens in all scripts and configuration files
- 4. **Test each script** — Run every script and verify it completes successfully against SaaS
- 5. **Document the mapping** — Record which SaaS token replaces which Managed token

### Meta-Monitoring Considerations

If you monitor Dynatrace itself (meta-monitoring), the health checks change:

| Managed Health Check               | SaaS Equivalent                                        |
|------------------------------------|--------------------------------------------------------|
| Cluster node availability          | Not applicable — Dynatrace manages SaaS infrastructure |
| Cassandra/Elasticsearch health     | Not applicable                                         |
| ActiveGate connectivity to cluster | ActiveGate connectivity to SaaS endpoints              |
| OneAgent connectivity to cluster   | OneAgent connectivity to SaaS endpoints                |
| Disk space on cluster nodes        | Not applicable                                         |

| Managed Health Check      | SaaS Equivalent                                   |
|---------------------------|---------------------------------------------------|
| API endpoint availability | API endpoint availability (same concept, new URL) |

## Verify API Connectivity

Use a simple API call to confirm SaaS API access is working:

```
# Test SaaS API connectivity
curl -s -o /dev/null -w "%{http_code}" \
  "https://{tenant}.live.dynatrace.com/api/v2/entities?pageSize=1" \
  -H "Authorization: Api-Token {SAAS_TOKEN}"
# Expected: 200
```

```
# Verify token scopes
curl -s "https://{tenant}.live.dynatrace.com/api/v2/apiTokens/lookup" \
  -H "Authorization: Api-Token {SAAS_TOKEN}" \
  -H "Content-Type: application/json" \
  -d '{"token": "{SAAS_TOKEN}"}' | jq '.scopes'
```

## 7. Synthetic Monitor Migration

Synthetic monitors cannot be directly migrated from Managed to SaaS. They must be recreated because synthetic execution depends on ActiveGate infrastructure and location IDs that differ between environments.

### Migration by Monitor Type

| Monitor Type             | Migration Approach                                                   |
|--------------------------|----------------------------------------------------------------------|
| <b>HTTP monitors</b>     | Export configuration; recreate in SaaS with same URLs and assertions |
| <b>Browser monitors</b>  | Export Selenium/Synthetic scripts; import into new SaaS monitors     |
| <b>Browser clickpath</b> | Re-record or import clickpath scripts                                |

### Private Synthetic Locations

If you use private synthetic locations (monitoring from inside your network):

1. **Deploy Synthetic-enabled ActiveGates** in SaaS — these replace your Managed Synthetic ActiveGates
2. **Create new private locations** in SaaS pointing to the new ActiveGates
3. **Assign monitors** to the new private locations
4. **Test execution** — verify monitors run from the new locations

### Public Synthetic Locations

Public locations are available in both Managed and SaaS. Simply recreate the monitors and assign the same geographic locations.

## Verify Synthetic Monitors Are Executing

```
// Count synthetic monitors in SaaS – compare to your Managed inventory
smartscapeNodes "BROWSER_MONITOR"
| summarize monitorCount = count()

// Smartscape (preferred, verified 07/2026): dt.entity.synthetic_test maps to the BROWSER_MONITOR
// node (individual steps are a separate BROWSER_MONITOR_STEP node). HTTP monitors are HTTP_MONITOR,
// multi-protocol monitors NETWORK_AVAILABILITY_MONITOR, private locations SYNTHETIC_LOCATION. This
// corrects an earlier note here that claimed no Smartscape equivalent existed. Unlike ActiveGate,
// `fetch dt.entity.synthetic_test` does still work and remains a genuine fallback – it reads the
// classic entity store, which can retain entities Smartscape (live topology) no longer lists.
// Classic fallback: fetch dt.entity.synthetic_test | summarize monitorCount = count()
```

```
// List synthetic monitors with their names – verify expected monitors are present
smartscapeNodes "BROWSER_MONITOR"
| fields name, type
| sort name asc

// Smartscape (preferred, verified 07/2026): dt.entity.synthetic_test maps to the BROWSER_MONITOR
// node (individual steps are a separate BROWSER_MONITOR_STEP node). HTTP monitors are HTTP_MONITOR,
// multi-protocol monitors NETWORK_AVAILABILITY_MONITOR, private locations SYNTHETIC_LOCATION. This
// corrects an earlier note here that claimed no Smartscape equivalent existed. Unlike ActiveGate,
// `fetch dt.entity.synthetic_test` does still work and remains a genuine fallback – it reads the
// classic entity store, which can retain entities Smartscape (live topology) no longer lists.
```

```
// Verify synthetic execution results are being recorded
fetch events, from:-1h
| filter event.kind == "SYNTHETIC_TEST_STEP_RESULT"
| summarize executionCount = count()
| fieldsAdd status = if(executionCount > 0, then: "Synthetic monitors executing", else: "No synthetic
results – check monitor configuration")
```

## 8. Step Completion Checklist

Do not proceed to Step 7 (Enable) until all items are confirmed.

| Checkpoint                                                                               | Status                   |
|------------------------------------------------------------------------------------------|--------------------------|
| All dashboard tiles showing data                                                         | <input type="checkbox"/> |
| Dashboard ownership and internal links updated to SaaS URLs                              | <input type="checkbox"/> |
| Hardcoded entity IDs replaced with entity selectors                                      | <input type="checkbox"/> |
| All notification channels tested end-to-end (email, Slack, Teams, PagerDuty, ServiceNow) | <input type="checkbox"/> |
| Problem notification resolution events verified                                          | <input type="checkbox"/> |
| CI/CD pipeline integrations updated with SaaS URLs and new tokens                        | <input type="checkbox"/> |
| Deployment events appearing in SaaS after CI/CD trigger                                  | <input type="checkbox"/> |
| ServiceNow integration reconnected and verified                                          | <input type="checkbox"/> |
| Jira integration updated (if applicable)                                                 | <input type="checkbox"/> |
| CMDB synchronization pointing to SaaS APIs                                               | <input type="checkbox"/> |
| Extensions 1.0 rebuilt as Extensions 2.0 (or confirmed SaaS-compatible)                  | <input type="checkbox"/> |

| Checkpoint                                                            | Status |
|-----------------------------------------------------------------------|--------|
| Extensions 2.0 deployed on host-based ActiveGates                     | [ ]    |
| Cloud integrations (AWS, Azure, GCP) reconfigured and metrics flowing | [ ]    |
| All API scripts updated with SaaS URLs and new tokens                 | [ ]    |
| Meta-monitoring health checks updated for SaaS architecture           | [ ]    |
| Synthetic monitors recreated and executing                            | [ ]    |
| Synthetic private locations deployed (if applicable)                  | [ ]    |

---

## Next Step

**M2S-07: Step 7 — Enable** — Train users and communicate the migration to the organization. Deliver persona-based training, update runbooks and documentation, publish self-service resources, and establish support channels for the broader user base.

### Additional Resources

- [SaaS Upgrade Assistant Documentation](#)
- [Automations \(Workflows\) Documentation](#)
- [Extensions 2.0 Documentation](#)
- [Cloud Integrations Overview](#)
- [Synthetic Monitoring Documentation](#)
- [Account Management API](#)

---

## Summary

In Step 6, you:

- Validated dashboard ownership, internal links, and entity references in SaaS
- Reconnected and tested all alerting notification channels end-to-end
- Updated CI/CD pipeline integrations with SaaS URLs and new API tokens
- Reconfigured ITSM integrations (ServiceNow, Jira, CMDB synchronization)
- Migrated extensions from EF1 to EF2 and reconfigured cloud integrations
- Updated all API scripts and meta-monitoring health checks for SaaS
- Recreated synthetic monitors with appropriate private and public locations

**Key Takeaway:** Integration is where the migration becomes real for everyone outside the Dynatrace admin team. Developers see deployment events, operations see alerts, and management sees dashboards. Test every channel before declaring Step 6 complete.

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*